Enlaces útiles · Sesión 3

Modelos de IA generativa open source

Aquí tenéis los enlaces que os van a hacer falta para la entrega final, ordenados por ejercicio, y unos cuantos más para ir más allá. No hace falta leerlos enteros: tirad de ellos cuando os atasquéis. Todos son documentación oficial y gratuita.

Revisados el 19 de octubre de 2026. Si alguno cambia, buscad la página equivalente en la misma web.

Para los ejercicios 1 y 2 (el RAG)

1 · Embeddings con Ollama y bge-m3

https://docs.ollama.com/capabilities/embeddings

https://ollama.com/library/bge-m3

Para qué sirve: Cómo se pide un vector a Ollama y el modelo de embeddings que usamos (multilingüe, 1.024 números).

Qué debéis saber hacer: Crear el índice y, para el ejercicio 1, probar otro modelo de embeddings.

2 · Búsqueda por palabras y fusión de listas

https://en.wikipedia.org/wiki/Okapi_BM25

https://cormack.uwaterloo.ca/cormacksigir09-rrf.pdf

Para qué sirve: BM25 (la búsqueda por palabras clave) y el artículo original de Reciprocal Rank Fusion (la híbrida).

Qué debéis saber hacer: Explicar por qué la híbrida combina posiciones y no puntuaciones.

3 · Vectores en DuckDB y recuperación con contexto

https://duckdb.org/docs/stable/sql/functions/list

https://www.anthropic.com/engineering/contextual-retrieval

Para qué sirve: Las funciones de listas de DuckDB (list_cosine_similarity, la que compara vectores) y un artículo sobre añadir contexto a cada fragmento antes de indexarlo, que es lo que hace nuestra cabecera.

Qué debéis saber hacer: Para la opción (a) del ejercicio 1: saber qué se pierde al quitar la cabecera.

Para el ejercicio 3 (el reto: el ticket)

4 · Imágenes con Ollama

https://docs.ollama.com/capabilities/vision

https://ollama.com/library/qwen3-vl/tags

Para qué sirve: Cómo se le pasa una imagen a un modelo de visión y las variantes de Qwen3-VL (fijaos en las «instruct» y en lo que ocupa cada una).

Qué debéis saber hacer: Descargar un modelo de visión que quepa en vuestro equipo y hacerle una pregunta sobre una imagen.

5 · Salidas estructuradas

https://docs.ollama.com/capabilities/structured-outputs

Para qué sirve: El parámetro format con un esquema JSON: el modelo solo puede escribir algo con esa forma. Trae un ejemplo con Pydantic, como el nuestro.

Qué debéis saber hacer: Distinguir pedir JSON en el prompt de obligarlo con un esquema.

6 · Pydantic

https://pydantic.dev/docs/validation/latest/get-started/

https://pydantic.dev/docs/validation/latest/concepts/json_schema/

Para qué sirve: Los modelos de datos con los que validamos lo que extrae el modelo (Ticket, Recuento, Albaran) y de los que sale el esquema JSON.

Qué debéis saber hacer: Leer el modelo Ticket de asistente/vision.py y saber qué rechazaría.

7 · La ficha de Qwen3-VL

https://huggingface.co/Qwen/Qwen3-VL-8B-Instruct

Para qué sirve: Quién lo publica, qué sabe hacer, sus límites y su licencia.

Qué debéis saber hacer: Encontrar la licencia del modelo de visión que uséis.

Para el ejercicio 3 (el reto: la API)

8 · FastAPI

https://fastapi.tiangolo.com/es/tutorial/

https://fastapi.tiangolo.com/tutorial/handling-errors/

Para qué sirve: El tutorial (en español) y cómo devolver errores con su código HTTP, que es lo que hace nuestra API con los errores con nombre.

Qué debéis saber hacer: Leer una operación de asistente/api.py y saber qué entra, qué sale y qué errores puede dar.

9 · Qué es OpenAPI

https://www.openapis.org/what-is-openapi

Para qué sirve: El estándar del contrato que FastAPI genera solo y que vemos en /docs.

Qué debéis saber hacer: Explicar por qué un contrato legible por máquinas sirve a otros programas, no solo a personas.

10 · Códigos de estado HTTP

https://developer.mozilla.org/es/docs/Web/HTTP/Reference/Status

Para qué sirve: La referencia de MDN, en español: qué significan 200, 422, 503 y compañía.

Qué debéis saber hacer: Distinguir «la petición está mal» (4xx) de «falla el servidor o una pieza» (5xx).

11 · Clientes HTTP en Python

https://requests.readthedocs.io/en/latest/user/quickstart/

https://www.python-httpx.org/

Para qué sirve: requests es el clásico; httpx es parecido y además asíncrono (es el que usa nuestra Pipe para no bloquear Open WebUI).

Qué debéis saber hacer: Hacer un POST con JSON y leer la respuesta.

12 · Consultas con parámetros

https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

Para qué sirve: La guía de OWASP contra la inyección de SQL: por qué un valor nunca debe pegarse dentro del texto de la consulta.

Qué debéis saber hacer: Explicar por qué «1; DROP TABLE ventas» no hace nada en nuestra API.

Para el ejercicio 4 (los PDF de junio)

13 · PaddleOCR

https://github.com/PaddlePaddle/PaddleOCR

https://www.paddleocr.ai/latest/en/version3.x/pipeline_usage/OCR.html

Para qué sirve: El proyecto (OCR, PP-StructureV3 y PaddleOCR-VL) y cómo se usa el OCR (PP-OCRv6), con la confianza por línea.

Qué debéis saber hacer: Saber qué devuelve un OCR y qué no.

14 · PaddleOCR-VL y el motor de inferencia

https://www.paddleocr.ai/latest/en/version3.x/pipeline_usage/PaddleOCR-VL.html

https://www.paddleocr.ai/latest/en/version3.x/inference_deployment/local_inference/inference_engine.html

Para qué sirve: El parser de documentos que usamos (maquetación + un VLM de 0,9 B) y cómo elegir el motor («transformers»: PyTorch, que en el Jetson usa la GPU).

Qué debéis saber hacer: Explicar por qué en el Jetson va en la GPU sin instalar PaddlePaddle.

15 · PyMuPDF

https://pymupdf.readthedocs.io/en/latest/

Para qué sirve: La librería con la que sacamos la capa de texto de un PDF y pasamos cada página a imagen.

Qué debéis saber hacer: Comprobar si un PDF tiene texto dentro o es solo imagen.

Para el ejercicio 5 (opcional, hacia producción)

16 · Seguridad en FastAPI

https://fastapi.tiangolo.com/tutorial/security/

Para qué sirve: Cómo pedir una clave o un usuario antes de atender una petición, con ejemplos que se copian y se prueban en /docs.

Qué debéis saber hacer: Que la API rechace (401) una petición sin clave y atienda la que la lleva.

17 · Desplegar con contenedores

https://fastapi.tiangolo.com/deployment/docker/

https://docs.docker.com/compose/

Para qué sirve: Cómo meter una aplicación FastAPI en un contenedor y arrancar varias piezas juntas (API, Open WebUI…) con un solo archivo de Docker Compose.

Qué debéis saber hacer: Explicar qué ganaríamos frente a arrancar cada pieza a mano en una terminal.

18 · Usuarios y permisos en Open WebUI

https://docs.openwebui.com/features/authentication-access/rbac/

Para qué sirve: Roles, grupos y permisos: quién puede usar qué modelo. En clase lo teníamos desactivado (WEBUI_AUTH=False).

Qué debéis saber hacer: Decir qué cambiaríais para que cada persona solo vea lo suyo.

19 · Registrar lo que pasa

https://docs.python.org/es/3/howto/logging.html

https://jsonlines.org/

Para qué sirve: El módulo logging de Python (en español) y el formato JSON Lines: una línea JSON por evento, fácil de guardar y de analizar luego con DuckDB.

Qué debéis saber hacer: Guardar cada pregunta con su traza y sus tiempos, y contarlas después con una consulta.

Para entender cómo funciona por dentro (opcional)

20 · Qué es un modelo de visión y lenguaje (VLM)

https://huggingface.co/blog/vlms

https://arxiv.org/abs/2010.11929

Para qué sirve: Una introducción clara a los VLM (codificador de imagen + proyector + LLM) y el artículo que convirtió la imagen en «palabras» de 16 × 16 píxeles (Vision Transformer).

Qué debéis saber hacer: Explicar por qué una imagen más grande cuesta más tokens y por qué un VLM puede inventar con aplomo.

21 · Qwen3-VL, el VLM que usamos

https://github.com/QwenLM/Qwen3-VL

Para qué sirve: El repositorio oficial, con ejemplos de OCR y de documentos («cookbooks»).

Qué debéis saber hacer: Saber qué tamaños hay y para qué está entrenado.

22 · PaddleOCR-VL, el parser de documentos

https://huggingface.co/PaddlePaddle/PaddleOCR-VL-1.6

https://arxiv.org/abs/2510.14528

Para qué sirve: La ficha del modelo (0,9 B, licencia Apache 2.0) y el artículo que explica sus dos pasos: maquetación y lectura de cada bloque.

Qué debéis saber hacer: Explicar por qué un parser devuelve tablas y un OCR, líneas.

23 · bge-m3 y el RAG original

https://huggingface.co/BAAI/bge-m3

https://arxiv.org/abs/2005.11401

Para qué sirve: El modelo de embeddings que usamos (multilingüe) y el artículo que puso nombre al RAG en 2020.

Qué debéis saber hacer: Situar nuestro RAG: qué es igual que en el artículo y qué hemos añadido (fichas, versiones, verificación).

Para ir más allá: Open WebUI

24 · Docker

https://docs.docker.com/get-started/

https://docs.docker.com/desktop/features/networking/networking-how-tos/

Para qué sirve: Qué es un contenedor y cómo se ve la red desde dentro en Docker Desktop (Mac y Windows), que no es igual que en Linux.

Qué debéis saber hacer: Arrancar un contenedor y saber cómo llega a un programa que corre en vuestro equipo.

25 · Open WebUI y sus Pipes

https://docs.openwebui.com/getting-started/quick-start/

https://docs.openwebui.com/features/extensibility/plugin/functions/pipe/

https://docs.openwebui.com/license/

Para qué sirve: Cómo se instala, qué es una Pipe (un «modelo» más del selector que en realidad es vuestro código) y su licencia, que no es MIT: tiene una cláusula de marca.

Qué debéis saber hacer: Instalar la Pipe y explicar qué hace ella y qué hace la API.

Para ir más allá: el flujo

26 · LangGraph

https://docs.langchain.com/oss/python/langgraph/overview

https://docs.langchain.com/oss/python/langgraph/graph-api

Para qué sirve: Qué es y su forma de describir un proceso: estado, nodos y aristas, lo mismo que hay en asistente/flujo.py.

Qué debéis saber hacer: Señalar en nuestro grafo el estado, un nodo y la función que decide el siguiente paso.

27 · Memoria, carga y contexto en Ollama

https://docs.ollama.com/faq

https://docs.ollama.com/context-length

Para qué sirve: Cuánto tiempo se queda un modelo cargado (keep_alive) y qué es el tamaño del contexto (por petición se cambia con num_ctx, en la API de chat). Lo de los 25 s de más al cambiar num_ctx entre pasos lo medimos nosotros: Ollama volvía a cargar el modelo.

Qué debéis saber hacer: Ver qué modelos están cargados y por qué a veces la primera pregunta tarda más.

Para ir más allá (opcional)

28 · La API como herramienta de Open WebUI

https://docs.openwebui.com/features/extensibility/plugin/tools/openapi-servers/

Para qué sirve: Open WebUI puede ofrecer a un modelo las operaciones de una API a partir de su contrato OpenAPI. Ojo: conectar una herramienta no garantiza que el modelo la elija bien, como vimos con el analista de la sesión 2.

Qué debéis saber hacer: Explicar la diferencia entre una Pipe que siempre llama a la API y un modelo que decide si la llama.

29 · La API de Ollama por dentro

https://docs.ollama.com/api/chat

Para qué sirve: La petición que hace nuestro asistente: mensajes, imágenes en base64, format y opciones como num_ctx.

Qué debéis saber hacer: Reconocer cada campo en asistente/ollama.py.

30 · Licencias

https://www.apache.org/licenses/LICENSE-2.0

https://choosealicense.com/licenses/

Para qué sirve: La licencia Apache 2.0 (la de Qwen3 y Qwen3-VL) y una comparación sencilla de las licencias abiertas más comunes.

Qué debéis saber hacer: Distinguir pesos abiertos de licencia abierta y saber dónde mirar antes de usar un modelo en una empresa.